

1. Introdução aos Comandos DDL

O que é DDL?

DDL (Data Definition Language) é responsável pela definição da estrutura do banco de dados. Ela permite criar, alterar e excluir objetos do banco, como tabelas, índices, views, entre outros. Os comandos DDL são essenciais para configurar o ambiente de banco de dados e gerenciar sua estrutura.

Principais Comandos DDL

1.1 CREATE

O comando CREATE permite a criação de diversos objetos no banco de dados. Vamos explorar em detalhes o uso do CREATE com diferentes tipos de objetos.

1.1.1 Criando Tabelas

A criação de tabelas é uma das funções mais comuns do comando CREATE. Aqui estão algumas maneiras de utilizá-lo:

Exemplo 1: Criação de uma Tabela Simples

```
CREATE TABLE funcionarios (  
    id NUMBER PRIMARY KEY,  
    nome VARCHAR2(100) NOT NULL,  
    salario NUMBER(10, 2),  
    data_admissao DATE  
);
```

Exemplo 2: Criando uma Tabela com Colunas Compostas

```
CREATE TABLE projetos (  
    id NUMBER PRIMARY KEY,  
    nome VARCHAR2(150) NOT NULL,  
    descricao VARCHAR2(500),  
    inicio DATE,  
    fim DATE,  
    estado VARCHAR2(20) CHECK (estado IN ('Ativo', 'Concluído', 'Suspenso'))  
);
```

Exemplo 3: Tabela com Chave Estrangeira

```
CREATE TABLE departamentos (  
    id NUMBER PRIMARY KEY,  
    nome VARCHAR2(100) NOT NULL  
);
```

```
CREATE TABLE funcionarios (  
    id NUMBER PRIMARY KEY,  
    nome VARCHAR2(100) NOT NULL,  
    departamento_id NUMBER,  
    CONSTRAINT fk_departamento FOREIGN KEY (departamento_id) REFERENCES departamentos(id)  
);
```

Exemplo 4: Criando uma Tabela com Índice

```
CREATE TABLE clientes (  
    id NUMBER PRIMARY KEY,  
    nome VARCHAR2(100) NOT NULL,  
    email VARCHAR2(150),  
    telefone VARCHAR2(15)  
);  
  
CREATE INDEX idx_clientes_email ON clientes(email);
```

1.1.2 Criando Views

Uma view é uma tabela virtual baseada no resultado de uma consulta SQL.

Exemplo: Criando uma View para Funcionários com Salário Acima de um Valor

```
CREATE VIEW funcionarios_altos_salarios AS  
  
SELECT id, nome, salario  
  
FROM funcionarios  
  
WHERE salario > 5000;
```

1.1.3 Criando Índices em Oracle

Os índices são estruturas que melhoram a velocidade das operações de recuperação de dados em tabelas. Eles são especialmente úteis quando você está lidando com grandes volumes de dados. Dos vários tipos de índices disponíveis no Oracle, o mais comum é o índice B-tree.

1. O que é um Índice?

Um índice é uma cópia ordenada de parte das colunas de uma tabela, que permite ao banco de dados localizar dados rapidamente sem ter que pesquisar em toda a tabela. Quando um índice é criado, o Oracle usa essa estrutura para acelerar as consultas.

2. Quando Criar um Índice?

- Consultas Frequentes: Se uma coluna é frequentemente usada em cláusulas WHERE, JOIN, ou ORDER BY, vale a pena criar um índice para melhorar o desempenho.

- **Tabelas Grandes:** Em tabelas que possuem um grande número de registros, os índices são essenciais, pois ajudam a reduzir o tempo de busca.
- **Colunas Únicas:** Colunas que contêm valores únicos, como chaves primárias ou colunas onde a propriedade UNIQUE é aplicada, devem ter índices para garantir a integridade dos dados.
- **Relatórios e Análises:** Se a tabela é usada para relatórios que precisam ordenar ou filtrar dados frequentemente, a criação de índices pode aumentar a eficiência.

3. Quando Não Criar um Índice?

- **Tabelas Pequenas:** Se a tabela possui um pequeno número de registros (por exemplo, menos de 1000), o custo de manter o índice pode superar os benefícios de desempenho.
- **Colunas de Alta Cardinalidade:** Colunas que têm muitos valores duplicados (exemplo: um campo Sexo com valores 'M' e 'F') podem não se beneficiar de um índice, pois o banco de dados ainda terá que examinar uma grande parte da tabela.
- **Operações DML Com Frequência:** Se a tabela é frequentemente atualizada, inserida ou excluída, a manutenção dos índices pode afetar negativamente o desempenho das operações DML (Data Manipulation Language).
- **Consultas que Retornam a Maioria dos Registros:** Se a consulta retorna a maioria dos registros da tabela, o uso de um índice pode não fornecer benefícios significativos.

4. Tipos de Índices

- **Índice B-tree:** O tipo mais comum, adequado para a maioria das buscas.
- **Índice Bitmap:** Útil para colunas com poucos valores distintos, como status (ativo/inativo). Normalmente usado em data warehouses.
- **Índices Compostos:** Índices que envolvem mais de uma coluna. Eles são úteis quando você costuma consultar várias colunas em conjunto.

5. Exemplos de Criação de Índices

Criando um Índice Simples:

```
CREATE INDEX idx_funcionarios_nome ON funcionarios(nome);
```

Este índice melhora a busca por nome na tabela funcionarios.

Criando um Índice Composto:

```
CREATE INDEX idx_funcionarios_departamento_salario ON funcionarios(departamento_id, salario);
```

Esse índice pode ser útil para consultas que filtram por departamento e ordenam por salário.

Criando um Índice Único:

```
CREATE UNIQUE INDEX idx_unico_email ON funcionarios(email);
```

Esse índice garante que não haja emails duplicados na tabela funcionarios.

Criando um Índice Bitmap:

```
CREATE BITMAP INDEX idx_funcionarios_sexo ON funcionarios(sexo);
```

Esse índice pode ser útil se a tabela funcionarios possui muitos registros, mas apenas dois valores diferentes para a coluna sexo.

6. Mantendo Índices

Os índices requerem espaço adicional em disco e devem ser mantidos. Quando os dados da tabela são alterados, inseridos ou excluídos, os índices devem ser atualizados conforme necessário. Por isso, você deve monitorar regularmente o desempenho e decidir se os índices existentes ainda são úteis ou se precisam ser ajustados.

7. Excluindo Índices

Se você determinar que um índice não está mais funcionando como esperado ou está desatualizado, você pode removê-lo para economizar espaço e melhorar a eficiência de DML.

Exemplo: Removendo um Índice:

```
DROP INDEX idx_funcionarios_nome;
```

Considerações Finais sobre o Uso de Índices

Os índices são uma ferramenta poderosa para melhorar a performance de consultas no banco de dados, mas devem ser utilizados com cuidado. Avaliar as características do seu uso de dados e consulta é essencial para determinar onde aplicar o uso de índices. Lembre-se sempre de medir o impacto antes e depois de criar ou remover índices para entender o efeito nas operações de leitura e escrita.

Com uma boa estratégia de indexação, você pode otimizar significativamente o desempenho do seu banco de dados.

1.1.4 Criando Sequências

Sequências são objetos que geram números sequenciais, comumente usados para gerar valores únicos para chaves primárias.

Exemplo: Criando uma Sequência

```
CREATE SEQUENCE seq_funcionarios_id
```

```
START WITH 1
```

```
INCREMENT BY 1
```

```
NOCACHE;
```

Usando a Sequência:

```
INSERT INTO funcionarios (id, nome, salario) VALUES (seq_funcionarios_id.NEXTVAL, 'Carlos Silva', 4500.00);
```

1.1.5 Criando Sincronização com Trigger

Os triggers são procedimentos que são automaticamente executados em resposta a certos eventos em uma tabela.

Exemplo: Trigger para Gerar ID Automaticamente

```
CREATE OR REPLACE TRIGGER trg_auto_id_funcionarios  
BEFORE INSERT ON funcionarios  
FOR EACH ROW  
BEGIN  
    :NEW.id := seq_funcionarios_id.NEXTVAL;  
END;
```

1.2 ALTER

O comando ALTER é utilizado para modificar a estrutura de objetos já existentes no banco de dados.

Adicionar uma Coluna:

```
ALTER TABLE funcionarios ADD funcionario_email VARCHAR2(100);
```

Modificar uma Coluna:

```
ALTER TABLE funcionarios MODIFY salario NUMBER(8, 2);
```

Remover uma Coluna:

```
ALTER TABLE funcionarios DROP COLUMN funcionario_email;
```

Adicionar uma Constraint:

```
ALTER TABLE funcionarios ADD CONSTRAINT chk_salario CHECK (salario > 0);
```

1.3 DROP

O comando DROP remove objetos do banco de dados.

Excluindo Tabelas:

```
DROP TABLE funcionarios CASCADE CONSTRAINTS;
```

Excluindo Views:

```
DROP VIEW funcionarios_altos_salarios;
```

Excluindo Índices:

```
DROP INDEX idx_funcionarios_nome;
```

1.4 TRUNCATE

O comando TRUNCATE remove todos os registros de uma tabela, mantendo a estrutura.

Exemplo:

```
TRUNCATE TABLE funcionarios;
```

1.5 RENAME

O comando RENAME é utilizado para renomear tabelas ou colunas.

Renomeando uma Tabela:

RENAME funcionarios TO empregados;

Renomeando uma Coluna:

ALTER TABLE empregados RENAME COLUMN nome TO nome_completo;

2. Introdução aos Comandos DCL

O que é DCL?

DCL, ou Data Control Language, refere-se a comandos SQL que controlam o acesso aos dados dentro de um banco de dados. Isso é crucial para garantir a segurança e a integridade dos dados.

Principais Comandos DCL

2.1 GRANT

O comando GRANT é utilizado para conceder permissões a usuários ou roles no banco de dados.

Exemplo de Concessão de Permissões:

GRANT SELECT, INSERT ON funcionarios TO usuario1;

- Isso concede ao usuario1 permissão para selecionar e inserir dados na tabela funcionarios.

2.2 REVOKE

O comando REVOKE é utilizado para revogar permissões que foram previamente concedidas.

Exemplo de Revogação de Permissões:

REVOKE INSERT ON funcionarios FROM usuario1;

- O usuario1 perde a permissão para inserir dados na tabela funcionarios.

3. Segurança de Banco de Dados e Roles

Importância da Segurança em Bancos de Dados

A segurança em bancos de dados é essencial para proteger informações sensíveis e evitar acessos não autorizados que podem levar à perda ou roubo de dados. Assegurar que apenas usuários autorizados tenham acesso a determinadas informações protege a integridade do sistema.

Princípios de Segurança

Princípio do Menor Privilégio:

Cada usuário deve ter apenas as permissões necessárias para realizar suas tarefas. Isso reduz o risco de acesso indevido.

Autenticação e Autorização:

- **Autenticação:** Processo de verificar a identidade de um usuário.

- **Autorização:** Processo de determinar se um usuário autenticado tem permissão para acessar um recurso.

Roles no Oracle

Uma *role* é um conjunto de permissões que pode ser atribuída a um ou mais usuários. Isso simplifica a concessão de permissões e facilita o gerenciamento de segurança.

Criando uma Role:

```
CREATE ROLE gerente_de_vendas;
```

Concedendo Permissões a uma Role:

```
GRANT SELECT, INSERT ON funcionarios TO gerente_de_vendas;
```

Atribuindo uma Role a um Usuário:

```
GRANT gerente_de_vendas TO usuario1;
```

Revogando uma Role de um Usuário:

```
REVOKE gerente_de_vendas FROM usuario1;
```

Considerações Finais

- **Comandos DDL** são usados para definir a estrutura do banco de dados, enquanto **comandos DCL** são usados para controlar o acesso aos dados.
- Compreender a importância de uma gestão de segurança eficaz é crucial para a proteção dos dados no banco de dados.
- O uso de *roles* facilita a atribuição de permissões e a gestão de segurança.

Dicas:

- Sempre faça um backup antes de utilizar comandos que podem modificar ou excluir dados, como DROP ou TRUNCATE.
- Utilize ALTER e CREATE com cuidado para não perder dados importantes.

Referências:

- [Oracle Database Documentation](#)